SQL DBA CHAMPS
Technical Knowledge Base Series

MySQL

Complete Technical Knowledge Base

From Fundamentals to Production Operations — architecture, installation, security, performance, high availability, disaster recovery, and troubleshooting.

Document Code	SDC-KB-MYSQL-001
Version	1.0
Prepared by	Praveen Madupu — Senior SQL Server & Cloud DBA
Applies to	MySQL 8.4 LTS (Community & Enterprise); notes on 8.0 EOL & 9.x Innovation
Date	July 14, 2026
Classification	Technical Reference — Internal / Training

© 2026 SQL DBA Champs. Prepared as an educational technical reference.

Table of Contents

1. Overview 4
1.1 A Brief History 4
1.2 Editions 4
1.3 Release Model & Version Guidance (2026) 4
1.4 MySQL vs. Alternatives 5
1.5 When to Use — and Not Use — MySQL 5
2. Architecture 6
2.1 Connection / Client Layer 6
2.2 SQL Layer (Server) 7
2.3 Pluggable Storage-Engine Layer 8
2.4 Process & Thread Model 9
2.5 InnoDB Internal Architecture 9
3. Components 10
3.1 Core Programs & Utilities 10
3.2 Key Files & Directories 10
3.3 Schema Objects 10
4. Installation 11
4.1 Ubuntu / Debian (APT, MySQL 8.4 LTS) 11
4.2 RHEL / Rocky / Oracle Linux (YUM/DNF) 11
4.3 Docker (labs / CI) 11
4.4 Secure the Fresh Install 11
5. Configuration 12
5.1 Key Server Variables 12
5.2 Sample Production [mysqld] (8.4 LTS, ~32 GB RAM host) 12
5.3 Runtime Changes with SET PERSIST 13
6. Prerequisites 14
6.1 Hardware Sizing (starting points) 14
6.2 Operating-System Tuning (Linux) 14
6.3 Software & Environment Checklist 14
7. Security Best Practices 15
7.1 Network Hardening 16
7.2 Authentication 16
7.3 Authorization — Least Privilege with Roles 16
7.4 Data Protection 16
7.5 Compliance Mapping (SOX / PCI-DSS) 17
8. Performance Tuning 17
8.1 Memory: the Buffer Pool First 17
8.2 Find the Expensive Queries 17
8.3 Read a Plan with EXPLAIN 18
8.4 Indexing Principles 18
8.5 InnoDB I/O & Durability Trade-offs 18
9. Monitoring 19

9.1 Golden Signals & Thresholds 19
9.2 Live Diagnostics from the Server 19
9.3 Tooling Options 20
10. Logging 21
10.1 Configure & Rotate 21
11. Backup & Recovery 22
11.1 Logical vs. Physical 22
11.2 Logical Backup Examples 22
11.3 Physical Backup with XtraBackup 23
11.4 Point-in-Time Recovery (PITR) 23
12. Disaster Recovery 24
12.1 RPO / RTO Tiers 24
12.2 InnoDB ClusterSet for Regional DR 24
12.3 DR Runbook Essentials 25
13. High Availability 26
13.1 Replication Fundamentals 26
13.2 InnoDB Cluster (recommended HA) 26
13.3 HA Options Compared 28
14. Troubleshooting 29
14.1 Scenario Playbook 29
14.2 Find and Break a Blocking Chain 29
14.3 Replication Health 29
15. Common Errors 30
16. FAQs 31
Q: Is MySQL free? 31
Q: InnoDB or MyISAM? 31
Q: What is the default isolation level? 31
Q: utf8 vs utf8mb4? 31
Q: Does MySQL support JSON? 31
Q: How do I scale reads? 31
Q: How do I scale writes? 31
Q: When should I shard? 31
Q: MySQL vs MariaDB — can I switch freely? 31
Q: How big can an InnoDB table get? 31
17. Interview Questions 32
17.1 Fundamentals 32
17.2 Performance 32
17.3 Replication, HA & Backup 32
18. Real-world Use Cases 34
19. Best Practices 35
19.1 Schema & Query 35
19.2 Configuration & Operations 35
19.3 Security, Backup & HA 35
20. References 36
20.1 Official Documentation 36
20.2 Tools 36

20.3 Recommended Reading 36

1. Overview

MySQL is the world's most widely deployed open-source relational database management system (RDBMS). It stores data in tables governed by a schema, is queried with SQL, and is engineered for high-throughput online transaction processing (OLTP). It powers a large share of the web — from small applications to hyperscale platforms such as Facebook, GitHub, Booking.com, Shopify, and Uber — and is a first-class citizen on every major cloud (Amazon RDS/Aurora, Azure Database for MySQL, Google Cloud SQL).

MySQL follows a **client-server** model. A single multi-threaded server process, **mysqld**, accepts connections from clients over TCP/IP or Unix sockets, parses and optimizes SQL, and delegates row storage to a **pluggable storage engine** — by default **InnoDB**, which provides ACID transactions, row-level locking, multi-version concurrency control (MVCC), crash recovery, and foreign keys.

1.1 A Brief History

Year	Milestone
1995	MySQL AB releases MySQL 1.0 (founders Widenius, Axmark, Larsson).
2005	InnoDB becomes the strategic transactional engine; MySQL 5.0 adds views, stored procedures, triggers.
2008-2010	Sun Microsystems acquires MySQL AB (2008); Oracle acquires Sun (2010). MariaDB is forked as a community alternative.
2018	MySQL 8.0 GA — data dictionary, CTEs, window functions, roles, atomic DDL, caching_sha2_password.
2023	Oracle introduces the Innovation + LTS release model (8.1 first Innovation release).
2024-2026	MySQL 8.4 LTS becomes the production default (support to 2032). MySQL 8.0 reaches End-of-Life April 2026. MySQL 9.x Innovation adds vector datatype and the HyperGraph optimizer.

1.2 Editions

Edition	License	What you get
MySQL Community	GPL v2 (free)	Full server + InnoDB, replication, Group Replication, MySQL Shell, Router. The basis of all cloud MySQL.
MySQL Enterprise	Commercial	Adds Enterprise Backup (MEB), TDE keyring, audit, firewall, data masking, Enterprise Monitor, and 24/7 support.
MySQL HeatWave	Cloud (OCI/AWS)	Managed MySQL with an in-memory analytics + ML accelerator for mixed OLTP/OLAP.

1.3 Release Model & Version Guidance (2026)

Since 2023 MySQL ships on two tracks: **LTS** (stable, ~2-year cadence, bug/security fixes only — safe for production) and **Innovation** (quarterly, newest features, ~8 months support). Choose one track per deployment.

Track / Version	Status (2026)	Guidance
MySQL 8.0	End-of-Life April 2026 (final 8.0.46)	Plan migration off 8.0 now; extended support only.
MySQL 8.4 LTS	Production default, supported to April 2032	Recommended for all new production clusters.
MySQL 9.x Innovation	Latest features (vector, HyperGraph optimizer)	Use for experimentation / feature adoption; short support window.

BEST PRACTICE

For new production systems in 2026, standardize on **MySQL 8.4 LTS**. It is a drop-in successor to 8.0 with no breaking changes across the LTS window and the broadest cloud and tooling support.

1.4 MySQL vs. Alternatives

Dimension	MySQL	MariaDB	PostgreSQL
Governance	Oracle	MariaDB Foundation (fork of MySQL)	Community (PostgreSQL Global Dev Group)
Default engine	InnoDB	InnoDB / Aria; also Galera-native HA	Native MVCC (no pluggable engines)
Sweet spot	Web/OLTP, single-shard up to ~10 TB	MySQL-compatible OLTP, richer engines	Complex queries, extensibility, JSONB, GIS
Cloud support	Broadest (RDS, Aurora, Cloud SQL)	Good	Very good

1.5 When to Use — and Not Use — MySQL

Strong fit

- Web applications and REST/API back-ends with high read/write OLTP volume.
- E-commerce, SaaS multi-tenant, content management, and session/state stores.
- Read-scale architectures (one primary, many replicas) behind a load balancer.

Consider alternatives when

- Large-scale analytical/columnar workloads — pair with a warehouse (Snowflake, StarRocks, HeatWave).
- Heavily document-centric models — evaluate PostgreSQL JSONB or MongoDB.
- You need horizontal write scaling beyond a single node — evaluate Vitess or NDB Cluster.

2. Architecture

MySQL's server is organized as a set of layers. Requests flow top-down: the **connection layer** authenticates and manages threads, the **SQL layer** parses/optimizes/executes statements, and the **storage-engine layer** performs the actual row storage and retrieval against **physical files**. The storage engine is pluggable, which is MySQL's defining architectural trait.

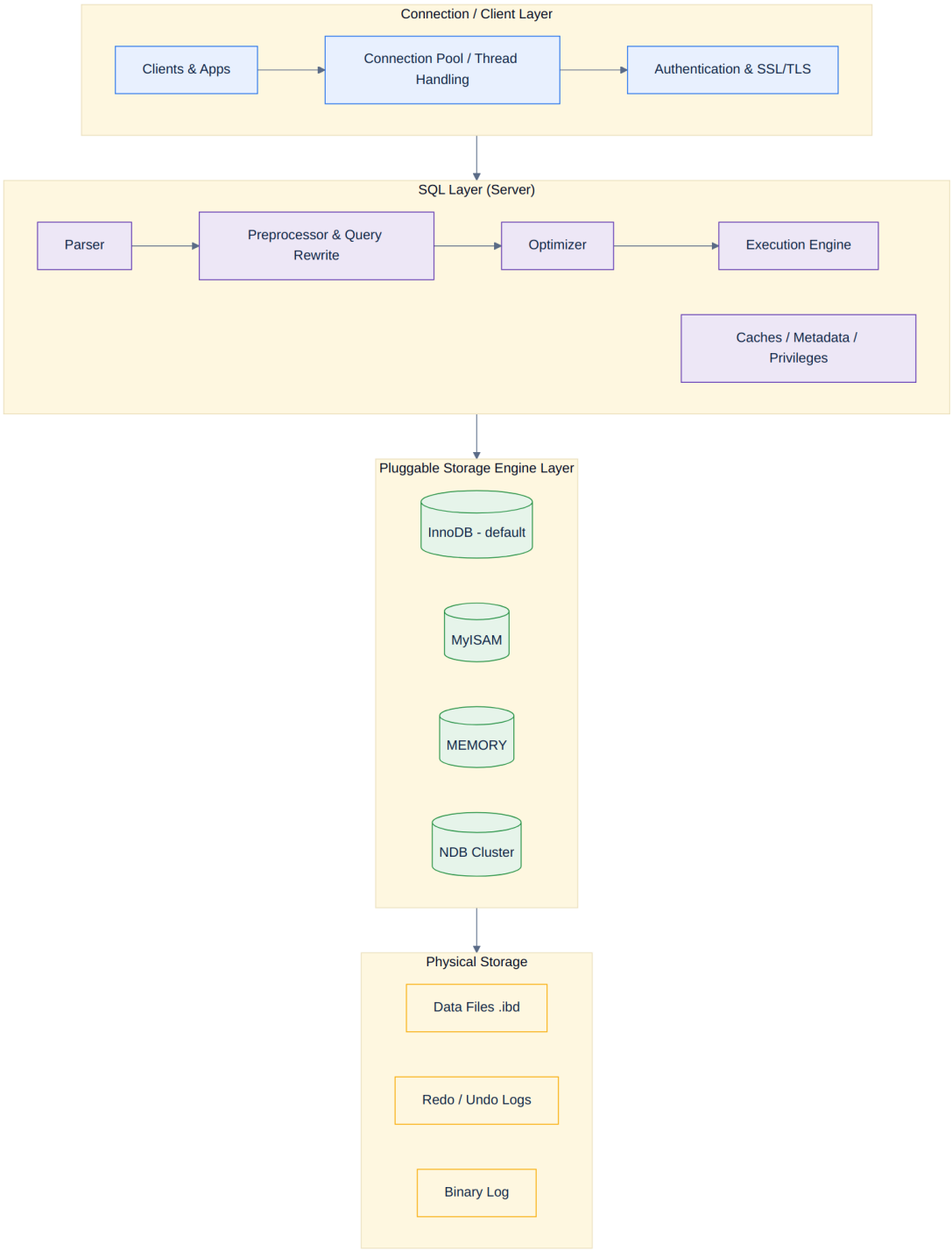


Figure 2.1 — MySQL layered logical architecture

2.1 Connection / Client Layer

Each client connection is handled by a dedicated OS thread (thread-per-connection model; an optional thread pool is available in Enterprise). This layer performs TCP/socket handling, TLS negotiation, authentication (default plugin **caching_sha2_password**), and connection-level session state.

2.2 SQL Layer (Server)

Statement pipeline

- **Parser** — tokenizes SQL and builds a parse tree; syntax errors surface here (ER 1064).
- **Preprocessor** — resolves table/column names, checks privileges, expands views.
- **Optimizer** — cost-based; chooses indexes, join order, and access methods. Since 9.x an optional HyperGraph optimizer improves complex JOIN planning.
- **Execution engine** — drives the plan by calling the storage-engine handler API row by row.
- **Caches & metadata** — the transactional data dictionary, privilege cache, and (removed in 8.0) the legacy query cache.

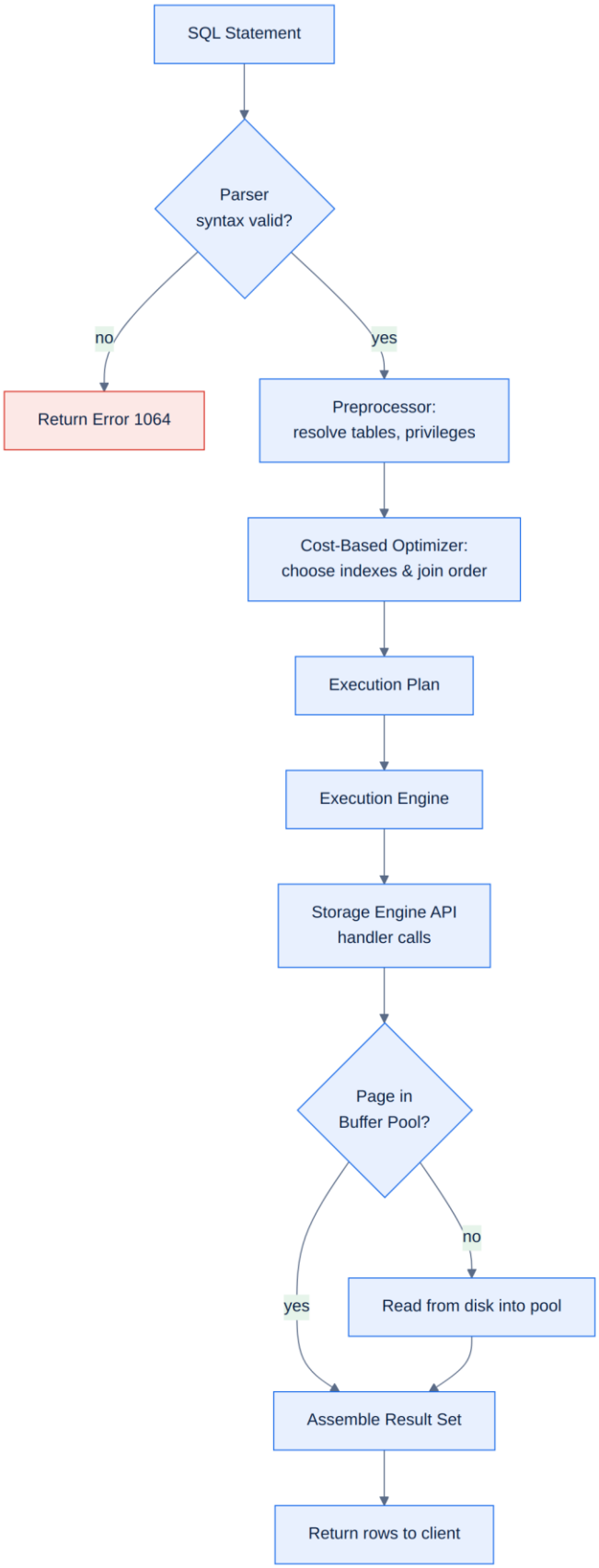


Figure 2.2 — Query execution flow (parse → optimize → execute → return)

2.3 Pluggable Storage-Engine Layer

Engine	Transactions	Locking	Use
InnoDB (default)	Yes (ACID)	Row-level, MVCC	Almost everything — OLTP, FKs, crash-safe.
MyISAM	No	Table-level	Legacy; read-mostly. Avoid for new work.
MEMORY	No	Table-level	Volatile temp/lookup tables in RAM.
NDB (Cluster)	Yes	Row-level	Shared-nothing, in-memory, auto-sharded HA.
ARCHIVE / CSV / FEDERATED	No	Varies	Niche: compressed history, CSV I/O, remote.

2.4 Process & Thread Model

MySQL is a single multi-threaded process. Key internal threads include the **connection threads**, InnoDB **background threads** (master, page cleaners, purge, IO), the **replication** dump/IO/applier threads, and the **event scheduler**. Understanding this model is essential for tuning `innodb_buffer_pool_instances`, page-cleaner threads, and the thread pool.

2.5 InnoDB Internal Architecture

InnoDB splits its structures into **in-memory** (buffer pool, change buffer, adaptive hash index, log buffer) and **on-disk** (system + file-per-table tablespaces, redo log, undo tablespaces, doublewrite buffer). Writes are journaled to the redo log first (write-ahead logging) and dirty pages are flushed asynchronously, giving durability without paying full random-write latency inline.

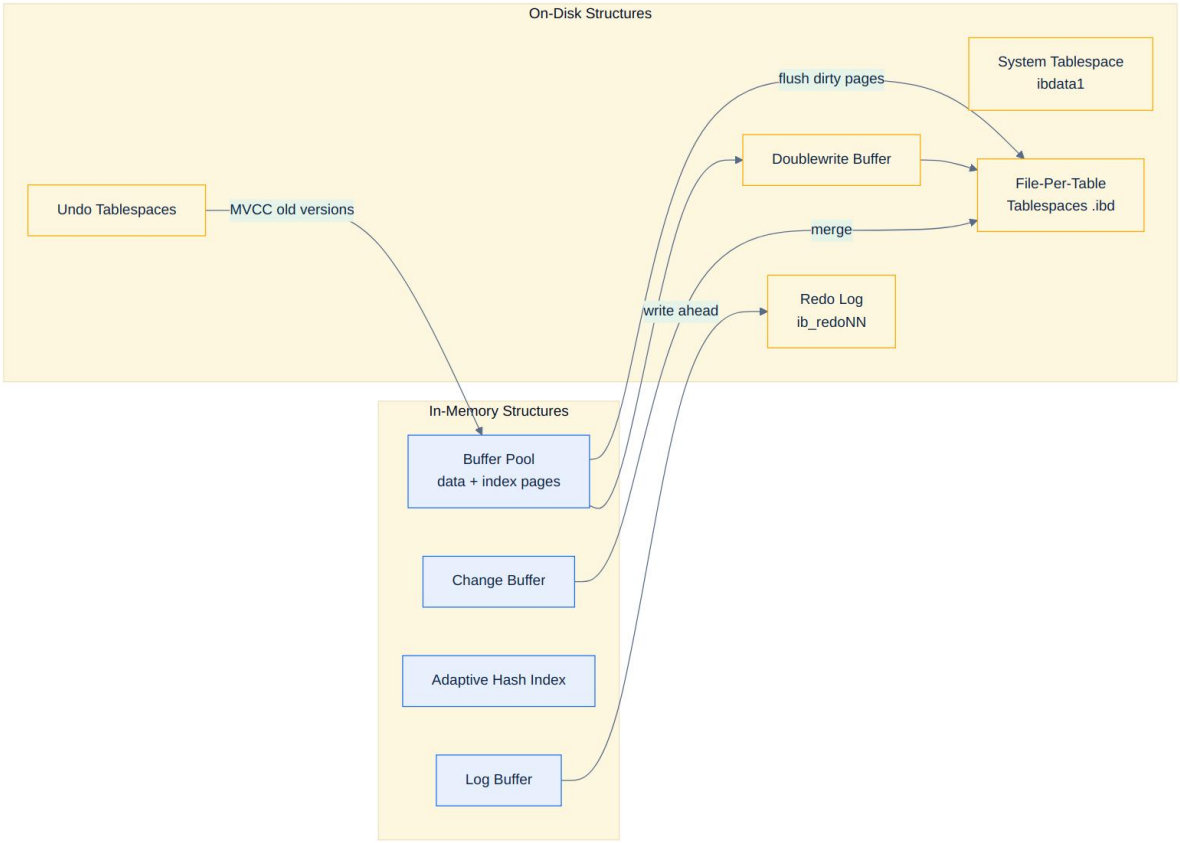


Figure 2.3 — InnoDB in-memory and on-disk structures

NOTE

The **buffer pool** is InnoDB's most important memory structure — it caches data and index pages. On a dedicated database host, size it to roughly **60–75% of RAM** (`innodb_buffer_pool_size`).

3. Components

3.1 Core Programs & Utilities

Component	Role
mysqld	The server daemon — the actual database process.
mysql	Interactive command-line SQL client.
mysqladmin	Administrative CLI (status, shutdown, flush, create/drop db).
mysqldump / mysqlpump	Logical backup — export schema + data as SQL (mysqlpump is parallel).
mysqlbinlog	Read/replay binary logs — the engine of point-in-time recovery.
MySQL Shell (mysqlsh)	Modern shell: SQL + JavaScript/Python, AdminAPI for InnoDB Cluster, dump/load utilities.
MySQL Router	Lightweight middleware that routes app traffic to the correct cluster node and handles failover.
MySQL Workbench	GUI for design, administration, and query.
mysqlcheck / myisamchk	Table maintenance — check, repair, analyze, optimize.

3.2 Key Files & Directories

Path / File	Purpose
/etc/my.cnf or /etc/mysql/my.cnf	Primary configuration file.
datadir (e.g. /var/lib/mysql)	All data: tablespaces, logs, system tables.
*.ibd	Per-table InnoDB data + indexes (file-per-table).
ibdata1	System tablespace (data dictionary metadata in older layouts, undo).
ib_redoNNN / #innodb_redo	Redo log files (write-ahead log).
undo_001, undo_002	Undo tablespaces (MVCC rollback + old row versions).
binlog.NNNNNN + binlog.index	Binary logs for replication and PITR.
error.log / slow.log	Diagnostics and slow-query capture.

3.3 Schema Objects

MySQL exposes four system schemas every DBA relies on: **information_schema** (SQL-standard metadata views), **performance_schema** (low-overhead runtime instrumentation), **sys** (human-friendly views built on performance_schema), and **mysql** (grant tables and server metadata). Practitioner queries throughout this document draw on these.

4. Installation

The recommended path on Linux is the **official MySQL APT/YUM repository**, which lets you pin an Innovation or LTS track. Docker suits labs and CI; tarballs suit air-gapped or multi-instance hosts.

4.1 Ubuntu / Debian (APT, MySQL 8.4 LTS)

```
# Add Oracle's MySQL APT repo and select the 8.4 LTS track
wget https://dev.mysql.com/get/mysql-apt-config_0.8.34-1_all.deb
sudo dpkg -i mysql-apt-config_0.8.34-1_all.deb   # choose "mysql-8.4-lts"
sudo apt-get update
sudo apt-get install -y mysql-community-server mysql-shell

sudo systemctl enable --now mysql
sudo systemctl status mysql
```

4.2 RHEL / Rocky / Oracle Linux (YUM/DNF)

```
sudo dnf install -y https://dev.mysql.com/get/mysql84-community-release-el9-
1.noarch.rpm
sudo dnf config-manager --disable mysql-innovation-community
sudo dnf config-manager --enable  mysql-8.4-lts-community
sudo dnf install -y mysql-community-server mysql-shell
sudo systemctl enable --now mysqld

# First-run root password (RHEL packages generate one)
sudo grep 'temporary password' /var/log/mysqld.log
```

4.3 Docker (labs / CI)

```
docker run --name mysql84 \
  -e MYSQL_ROOT_PASSWORD='StrongP@ss#2026' \
  -e MYSQL_DATABASE=appdb \
  -p 3306:3306 \
  -v mysql_data:/var/lib/mysql \
  -d mysql:8.4

docker exec -it mysql84 mysql -uroot -p
```

4.4 Secure the Fresh Install

```
# Interactive hardening: sets root auth, removes anon users,
# test db, and disables remote root login.
sudo mysql_secure_installation

# Verify
mysql -uroot -p -e "SELECT VERSION(); SHOW ENGINES\G"
```

WARNING

Never expose port **3306** to the public internet. Bind to a private interface (`bind-address`) and restrict

access with firewall/security-group rules. See Section 7.

5. Configuration

MySQL reads options from `my.cnf` (Linux) / `my.ini` (Windows) at startup, organized into groups like `[mysqld]`, `[client]`, and `[mysql]`. Most variables can also be changed at runtime; in 8.0+ use **`SET PERSIST`** to change a global variable and survive restarts (it writes to `mysqld-auto.cnf`).

5.1 Key Server Variables

Variable	Typical value	What it controls
innodb_buffer_pool_size	60-75% of RAM	Primary data/index cache — the #1 performance lever.
innodb_redo_log_capacity	2-8 GB (8.0.30+)	Redo log size; larger absorbs write bursts (replaces innodb_log_file_size).
innodb_flush_log_at_trx_commit	1 (2 for relaxed)	Durability vs throughput trade-off.
innodb_flush_method	O_DIRECT	Bypass OS cache to avoid double buffering.
innodb_io_capacity / _max	2000 / 4000 (SSD)	Background flush rate.
max_connections	200-1000	Concurrent client cap — right-size to avoid memory blowups.
sync_binlog	1	Flush binlog per commit for crash safety (with trx_commit=1 = full durability).
character_set_server	utf8mb4	Full Unicode incl. emoji; use with utf8mb4_0900_ai_ci.
max_allowed_packet	64M-256M	Largest single query/BLOB payload.

5.2 Sample Production [mysqld] (8.4 LTS, ~32 GB RAM host)

```
[mysqld]
# --- Networking ---
bind-address                = 10.0.1.20
max_connections             = 500
skip_name_resolve           = ON

# --- InnoDB core ---
innodb_buffer_pool_size     = 22G
innodb_buffer_pool_instances = 8
innodb_redo_log_capacity    = 4G
innodb_flush_log_at_trx_commit = 1
innodb_flush_method         = O_DIRECT
innodb_io_capacity          = 2000
innodb_io_capacity_max      = 4000

# --- Durability / replication ---
server_id                   = 1
gtid_mode                   = ON
enforce_gtid_consistency    = ON
log_bin                     = binlog
```

```
binlog_expire_logs_seconds   = 604800   # 7 days
sync_binlog                  = 1

# --- Charset & logging ---
character_set_server         = utf8mb4
collation_server             = utf8mb4_0900_ai_ci
slow_query_log               = ON
long_query_time              = 1
log_error_verbosity          = 2
```

5.3 Runtime Changes with SET PERSIST

```
-- Change and persist across restarts (writes mysqld-auto.cnf)
SET PERSIST max_connections = 800;
SET PERSIST innodb_io_capacity = 3000;

-- Persist a value already active, without changing the running value
SET PERSIST_ONLY innodb_buffer_pool_instances = 8;

-- Inspect what has been persisted
SELECT * FROM performance_schema.persisted_variables;
```

BEST PRACTICE

Prefer `SET PERSIST` for dynamic tuning and keep static/structural settings (`server_id`, `log_bin`, paths) in `my.cnf` under version control. Never hand-edit `mysqld-auto.cnf`.

6. Prerequisites

6.1 Hardware Sizing (starting points)

Tier	vCPU	RAM	Storage
Dev / lab	2	4-8 GB	SSD, 20-50 GB
Small production	4-8	16-32 GB	NVMe SSD, RAID10
Medium OLTP	8-16	64-128 GB	NVMe, separate data/redo volumes
Large OLTP	16-64	256 GB+	NVMe arrays, provisioned IOPS

NOTE
Memory is the dominant lever: aim to fit the working set in the InnoDB buffer pool. Prefer NVMe/SSD — InnoDB is latency-sensitive on redo and doublewrite writes.

6.2 Operating-System Tuning (Linux)

Setting	Recommended	Why
Transparent Huge Pages	Disabled	THP causes latency spikes / stalls for InnoDB.
vm.swappiness	1	Avoid swapping the buffer pool to disk.
Filesystem	XFS (or ext4)	XFS scales well for concurrent DB I/O.
I/O scheduler	none/noop (NVMe), mq-deadline	Let fast SSDs schedule themselves.
open files (nofile)	>= 65535	Many tablespaces + connections need FDs.
ulimit / LimitNOFILE	Raised in systemd unit	Prevent 'too many open files' errors.

```
# Disable THP at boot (systemd drop-in or kernel cmdline)
echo never > /sys/kernel/mm/transparent_hugepage/enabled

# Raise file descriptors for the mysqld unit
sudo mkdir -p /etc/systemd/system/mysqld.service.d
printf '[Service]\nLimitNOFILE=65535\n' | \
  sudo tee /etc/systemd/system/mysqld.service.d/limits.conf
sudo systemctl daemon-reload && sudo systemctl restart mysqld
```

6.3 Software & Environment Checklist

Before install

- 64-bit Linux (RHEL/Rocky 8-9, Ubuntu 22.04/24.04) or supported Windows Server.
- Dedicated `mysql` OS user and group; data directory on a dedicated volume.
- NTP/chrony time sync (critical for replication and audit correctness).
- SELinux/AppArmor in enforcing mode with correct MySQL contexts (do not simply disable).
- Network ports planned: 3306 (SQL), 33060 (X Protocol), 33061 (Group Replication).

7. Security Best Practices

Secure MySQL in defense-in-depth layers — network, authentication, authorization, and data protection — and prove it with auditing. The diagram below maps the layers a hardened deployment should implement.

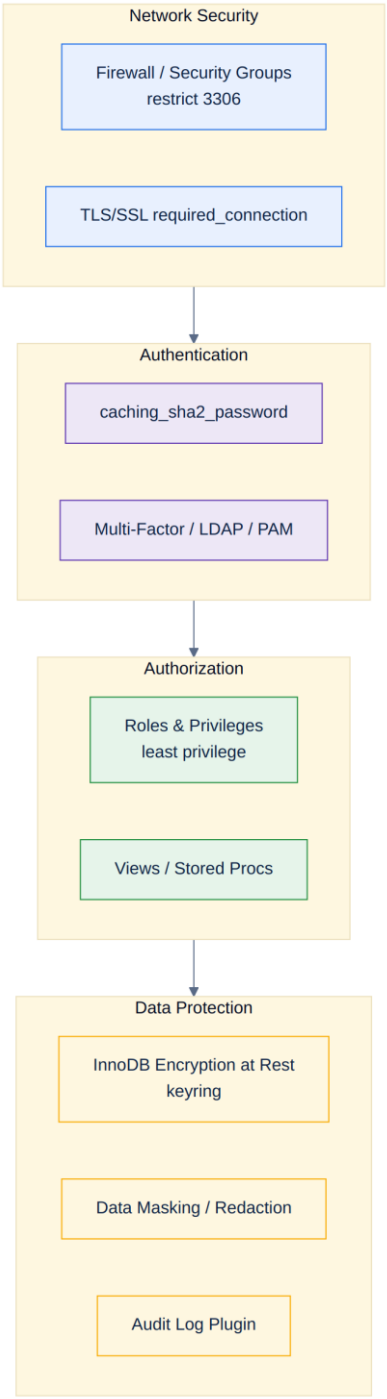


Figure 7.1 — MySQL security defense-in-depth layers

7.1 Network Hardening

Reduce the attack surface

- Bind to a private address (`bind-address`) and firewall port 3306 to app subnets only.
- Require TLS for all connections (`require_secure_transport = ON`).
- Set `skip_name_resolve = ON` to authenticate on IP, not DNS.
- Never run mysqld as root; run as the dedicated `mysql` user.

7.2 Authentication

```
-- Default strong plugin in 8.x
CREATE USER 'app'@'10.0.%'
  IDENTIFIED WITH caching_sha2_password BY 'S0me-Strong#Secret';

-- Enforce password strength + rotation policy
INSTALL COMPONENT 'file://component_validate_password';
SET GLOBAL validate_password.policy = STRONG;
ALTER USER 'app'@'10.0.%' PASSWORD EXPIRE INTERVAL 90 DAY;

-- Require TLS for a specific account
ALTER USER 'app'@'10.0.%' REQUIRE SSL;
```

7.3 Authorization — Least Privilege with Roles

```
CREATE ROLE 'app_rw', 'app_ro';
GRANT SELECT, INSERT, UPDATE, DELETE ON appdb.* TO 'app_rw';
GRANT SELECT ON appdb.* TO 'app_ro';

GRANT 'app_rw' TO 'app'@'10.0.%';
SET DEFAULT ROLE 'app_rw' TO 'app'@'10.0.%';

-- Never grant ALL PRIVILEGES or *.* to application users.
-- Audit privileges:
SELECT * FROM information_schema.USER_PRIVILEGES;
```

7.4 Data Protection

Control	Mechanism	Notes
Encryption at rest	InnoDB tablespace encryption + keyring	`ALTER TABLE ... ENCRYPTION='Y'`; manage keys via keyring_file/KMS.
Encryption in transit	TLS 1.2/1.3	Server certs + `require_secure_transport`.
Auditing	Enterprise Audit / MariaDB audit plugin	Log DDL/DML/logins to satisfy SOX/PCI evidence.
Data masking	Enterprise data masking / views	Redact PII for non-privileged roles.
Binlog encryption	binlog_encryption = ON	Protect replicated change stream at rest.

7.5 Compliance Mapping (SOX / PCI-DSS)

Requirement	MySQL control
Access control & least privilege	Roles, per-host accounts, no shared logins, quarterly access review.
Encryption of cardholder/PII data	TDE at rest + TLS in transit + column/app-level encryption for PANs.
Audit trail of privileged actions	Audit log plugin capturing logins, GRANTs, and DDL, shipped to a WORM store.
Change management	my.cnf under version control; SET PERSIST changes logged and reviewed.
Backup & recoverability	Encrypted, tested, offsite backups with documented RTO/RPO (Sections 11-12).

CRITICAL

Immediately after install: run `mysql_secure_installation`, drop anonymous users, remove the `test` database, disable remote root, and rotate the initial root password. These are the most common findings in a security audit.

8. Performance Tuning

Performance work proceeds top-down: right-size **memory**, then fix **queries and indexes**, then tune **InnoDB I/O and durability**, and finally address **concurrency and schema**. Measure with `performance_schema`/`sys` and the slow query log before and after every change — never tune blind.

8.1 Memory: the Buffer Pool First

```
-- How much of the working set is cached?
SELECT
  ROUND(100 * (1 - (
    (SELECT VARIABLE_VALUE FROM performance_schema.global_status
       WHERE VARIABLE_NAME='Innodb_buffer_pool_reads')
    / NULLIF((SELECT VARIABLE_VALUE FROM performance_schema.global_status
       WHERE VARIABLE_NAME='Innodb_buffer_pool_read_requests'),0)
  )), 2) AS buffer_pool_hit_pct;

-- Resize online (8.0+); allocation runs in the background
SET GLOBAL innodb_buffer_pool_size = 24 * 1024 * 1024 * 1024;
```

BEST PRACTICE

Target a buffer pool hit ratio **> 99%** for OLTP. Persistent misses mean the working set doesn't fit — add RAM or reduce scanned data with better indexes.

8.2 Find the Expensive Queries

```
-- Top statements by total latency (sys schema)
SELECT query, exec_count, total_latency, rows_examined_avg, rows_sent_avg
FROM sys.statement_analysis
ORDER BY total_latency DESC LIMIT 15;
```

```
-- Full-table scans and queries not using indexes
SELECT * FROM sys.statements_with_full_table_scans
ORDER BY total_latency DESC LIMIT 15;

-- Offline: aggregate the slow log
pt-query-digest /var/lib/mysql/slow.log > digest.txt
```

8.3 Read a Plan with EXPLAIN

```
EXPLAIN FORMAT=JSON
SELECT o.id, o.total
FROM orders o
WHERE o.customer_id = 42 AND o.status = 'PAID'
ORDER BY o.created_at DESC LIMIT 20;

-- Actual executed plan with real row counts & timing (8.0.18+)
EXPLAIN ANALYZE SELECT ...;
```

What to look for

- `type` = ALL means a full scan — usually add or fix an index.
- A composite index on `(customer_id, status, created_at)` can satisfy the filter and the ORDER BY together.
- Watch `rows_examined` vs `rows_sent` — a large gap signals wasted work.
- `Using filesort` / `Using temporary` on hot queries are red flags.

8.4 Indexing Principles

Principle	Guidance
Selective leading column	Put the most selective equality column first in composite indexes.
Covering indexes	Include all selected columns so the query is answered from the index alone.
Left-prefix rule	An index on (a,b,c) helps a, a+b, a+b+c — not b alone.
Avoid over-indexing	Every index slows writes and consumes buffer pool; drop unused ones.
Find unused indexes	sys.schema_unused_indexes lists candidates for removal.

8.5 InnoDB I/O & Durability Trade-offs

Parameter	Throughput setting	Safety setting
innodb_flush_log_at_trx_commit	2 (flush ~1/sec)	1 (flush per commit — ACID)
sync_binlog	0 / 1000	1 (crash-safe binlog)
innodb_io_capacity_max	Higher on NVMe	Match device to avoid checkpoint storms
innodb_redo_log_capacity	Larger absorbs bursts	Size for ~1h of redo

WARNING

Parameter	Throughput setting	Safety setting
`innodb_flush_log_at_trx_commit=2` and `sync_binlog=0` boost throughput but can lose up to ~1 second of committed transactions on OS/host crash. Only relax durability where the workload tolerates it.		

9. Monitoring

A production MySQL estate needs metrics (time-series), logs, and alerting. The common open-source stack exports metrics via **mysqld_exporter** into **Prometheus**, visualizes in **Grafana**, and alerts through **Alertmanager**. Percona **PMM** bundles this end-to-end; MySQL Enterprise Monitor is the commercial option.

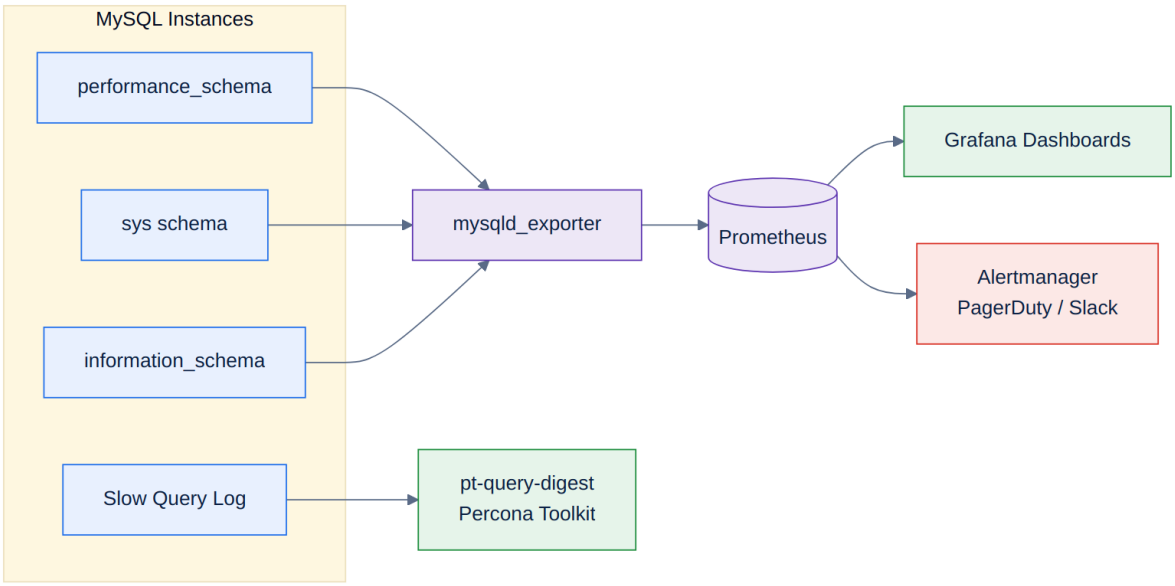


Figure 9.1 — Reference monitoring & alerting stack

9.1 Golden Signals & Thresholds

Metric	Source	Watch / Alert
Buffer pool hit ratio	Innodb_buffer_pool_read_requests/reads	< 99% (OLTP) — investigate
Queries per second (QPS)	Questions / Com_*	Baseline; alert on sharp anomalies
Threads_running	SHOW GLOBAL STATUS	> ~ (cores) sustained = contention
Replication lag	Seconds_Behind_Source	> a few seconds sustained
Connections used	Threads_connected / max_connections	> 80% of max_connections
InnoDB row lock waits	Innodb_row_lock_waits / time	Rising = lock contention
Aborted_connects	Global status	Spikes = auth/network issues
Disk free on datadir	OS	< 15% free — act before it fills

9.2 Live Diagnostics from the Server

```
-- Overall status and engine internals
SHOW GLOBAL STATUS LIKE 'Threads_%';
SHOW ENGINE INNODB STATUS\G      -- deadlocks, semaphores, I/O
```

```
-- Current activity and long-running statements
SELECT * FROM sys.processlist WHERE conn_id IS NOT NULL
  AND time > 5 ORDER BY time DESC;

-- Wait/latency hotspots
SELECT * FROM sys.wait_classes_global_by_latency;
```

9.3 Tooling Options

Tool	Type	Best for
Prometheus + mysqld_exporter + Grafana	OSS	Custom dashboards & alerting at scale.
Percona Monitoring & Management (PMM)	OSS	Turnkey MySQL/Postgres/Mongo monitoring + QAN.
MySQL Enterprise Monitor	Commercial	Advisors, query analyzer, official support.
Cloud native (RDS Perf Insights, Cloud SQL)	Managed	Zero-ops metrics on managed MySQL.

10. Logging

Log	Purpose	Key settings
Error log	Startup/shutdown, crashes, warnings	log_error, log_error_verbosity
Slow query log	Queries over a latency threshold	slow_query_log, long_query_time, log_queries_not_using_indexes
General query log	Every statement (debug only)	general_log — heavy; keep OFF in prod
Binary log	Change stream for replication + PITR	log_bin, binlog_format=ROW, binlog_expire_logs_seconds
Relay log	Replica's copy of received binlog events	relay_log, relay_log_recovery=ON
Redo / Undo	InnoDB crash recovery + MVCC	innodb_redo_log_capacity (internal)

10.1 Configure & Rotate

```
-- Enable the slow log and capture unindexed queries
SET GLOBAL slow_query_log = ON;
SET GLOBAL long_query_time = 1;
SET GLOBAL log_queries_not_using_indexes = ON;

-- Row-based binlog is required for safe HA/PITR
SET PERSIST binlog_format = 'ROW';
SET PERSIST binlog_expire_logs_seconds = 604800;    -- 7 days

-- Rotate logs safely
FLUSH BINARY LOGS;    -- start a new binlog
FLUSH ERROR LOGS;     -- reopen after logrotate moves the file
```

BEST PRACTICE

Use OS **logrotate** for the error/slow logs (with `FLUSH LOGS` post-rotate), and rely on `binlog_expire_logs_seconds` for automatic binlog purging. Ship logs to a central store (ELK/Loki) for retention and audit.

11. Backup & Recovery

A recoverable MySQL system combines **full** backups, optional **incrementals**, and a **continuous binary-log** stream to enable point-in-time recovery (PITR). Choose logical backups for portability and physical backups for speed at scale.

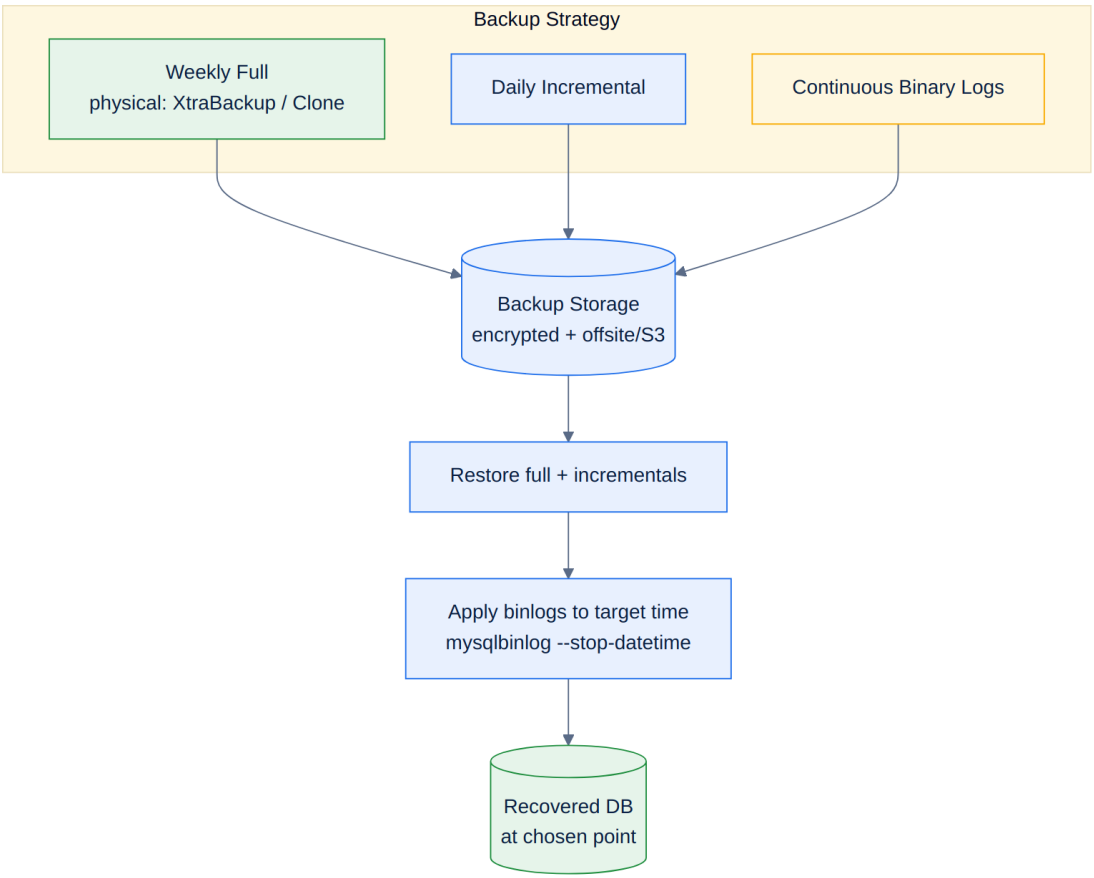


Figure 11.1 — Backup strategy with point-in-time recovery

11.1 Logical vs. Physical

Type	Tools	Pros / Cons
Logical	mysqldump, mysqlpump, MySQL Shell util.dumpInstance	Portable, selective, human-readable; slower to restore on big data.
Physical	Percona XtraBackup, Clone plugin, MySQL Enterprise Backup	Fast full/incremental at scale; version/engine-specific.
Snapshot	LVM / cloud volume snapshots	Instant if crash-consistent (FLUSH TABLES WITH READ LOCK or engine support).

11.2 Logical Backup Examples

```
# Consistent full logical backup (single transaction, GTID recorded)
mysqldump --single-transaction --source-data=2 --set-gtid-purged=ON \
```

```
  --routines --triggers --events --all-databases \
  | gzip > /backups/full_$(date +%F).sql.gz

# Fast parallel logical dump with MySQL Shell (recommended for large DBs)
mysqlsh root@localhost -- util dumpInstance /backups/dump_$(date +%F) \
  --threads=8 --compression=zstd
```

11.3 Physical Backup with XtraBackup

```
# Full physical backup
xtrabackup --backup --target-dir=/backups/full \
  --user=bkp --password='***'

# Prepare (apply redo) then restore into an empty datadir
xtrabackup --prepare --target-dir=/backups/full
systemctl stop mysqld
xtrabackup --copy-back --target-dir=/backups/full
chown -R mysql:mysql /var/lib/mysql
systemctl start mysqld
```

11.4 Point-in-Time Recovery (PITR)

Restore the most recent full backup, then replay binary logs up to the instant just before the incident (for example an accidental `DROP TABLE` at 14:05).

```
# 1) Restore the full backup (logical example)
zcat /backups/full_2026-01-12.sql.gz | mysql

# 2) Replay binlogs from the backup position up to just before the error
mysqlbinlog --start-position=195 \
  --stop-datetime="2026-01-12 14:04:59" \
  /var/lib/mysql/binlog.000045 /var/lib/mysql/binlog.000046 \
  | mysql
```

Backup discipline

- **Test restores** on a schedule — an untested backup is a hope, not a backup.
- Encrypt backups and store copies **offsite** (3-2-1 rule).
- Record the GTID/binlog position with every full backup to enable clean PITR.
- Automate verification: restore to a scratch host and run `mysqlcheck` + row-count checks.

12. Disaster Recovery

Disaster recovery (DR) extends backup/HA to survive the loss of an entire site or region. It is defined by two targets: **RPO** (how much data you can lose) and **RTO** (how long recovery may take). Architect and, crucially, **rehearse** to those numbers.

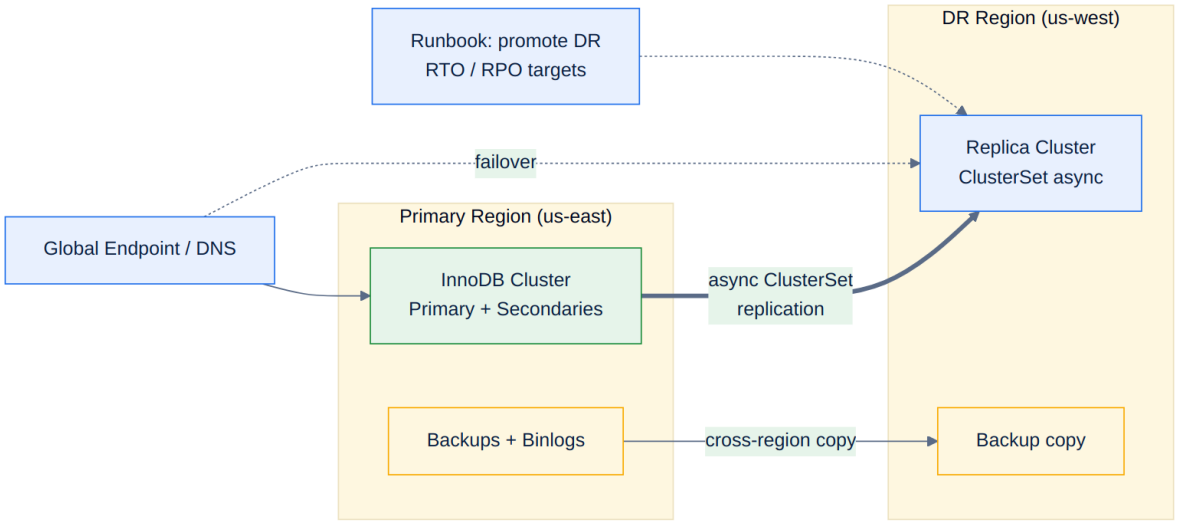


Figure 12.1 — Cross-region DR with InnoDB ClusterSet and offsite backups

12.1 RPO / RTO Tiers

Tier	RPO	RTO	Architecture
Basic	Hours	Hours-days	Nightly offsite backups + binlog shipping; manual rebuild.
Standard	Minutes	< 1 hour	Async replica in a second region; scripted promotion.
Advanced	Seconds	Minutes	InnoDB ClusterSet (primary cluster + async replica cluster) with Router failover.
Continuous	~Zero	Seconds	Multi-region synchronous / Aurora Global-style with automated failover.

12.2 InnoDB ClusterSet for Regional DR

```
-- On the primary cluster's shell session
myCluster  = dba.getCluster();
clusterSet = myCluster.createClusterSet('prodCS');

-- Attach a replica cluster in the DR region (async ClusterSet link)
clusterSet.createReplicaCluster('dr-node1:3306', 'drCluster');

-- Controlled regional failover (planned) / emergency failover
clusterSet.setPrimaryCluster('drCluster');
-- clusterSet.forcePrimaryCluster('drCluster');  // when primary is lost
```

12.3 DR Runbook Essentials

Every DR plan documents

- Declared RPO/RTO and the trigger criteria for invoking DR.
- Step-by-step promotion of the DR site, DNS/endpoint cutover, and app reconfiguration.
- Data-consistency verification (GTID gap check, row counts, checksums).
- Failback procedure to return to the primary region once healthy.
- A **tested** cadence — run a full DR game-day at least twice a year.

CRITICAL

Replication and HA are **not** a substitute for backups: a logical corruption or malicious DROP replicates instantly to every replica and region. Backups + PITR are your only recovery path from bad data.

13. High Availability

High availability (HA) keeps MySQL serving through node failures. Options range from simple asynchronous replication to fully automated **InnoDB Cluster** (Group Replication + MySQL Router + Shell AdminAPI). Replication also underpins read-scale-out.

13.1 Replication Fundamentals

The primary records committed changes to its **binary log**. Each replica's **IO thread** pulls those events into a **relay log**, and **applier threads** replay them. With **GTIDs**, every transaction carries a globally unique ID, making failover and re-pointing replicas reliable.

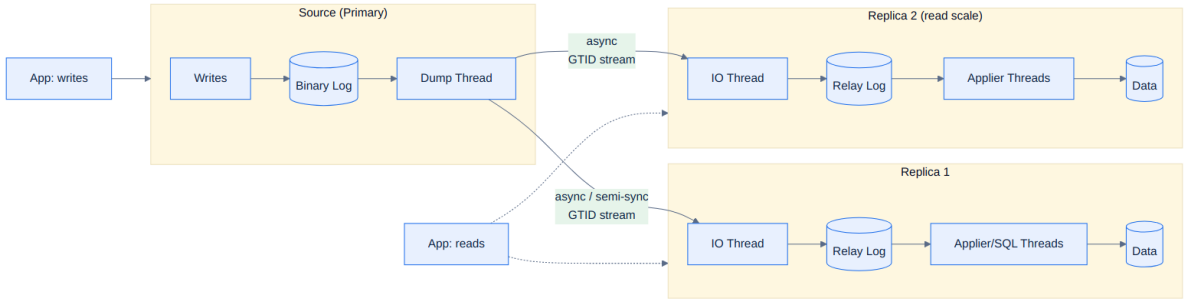


Figure 13.1 — Asynchronous / semi-synchronous replication topology

Mode	Durability	Notes
Asynchronous	Primary doesn't wait	Lowest latency; risk of data loss on primary failure.
Semi-synchronous	Waits for >=1 replica ack	Bounded data loss; slight latency cost (rpl_semi_sync).
Group Replication	Group consensus (Paxos)	Synchronous certification; foundation of InnoDB Cluster.

13.2 InnoDB Cluster (recommended HA)

InnoDB Cluster bundles Group Replication for data, MySQL Router for transparent routing/failover, and MySQL Shell's AdminAPI for lifecycle management — a self-healing, single-primary (or multi-primary) cluster with automatic leader election.

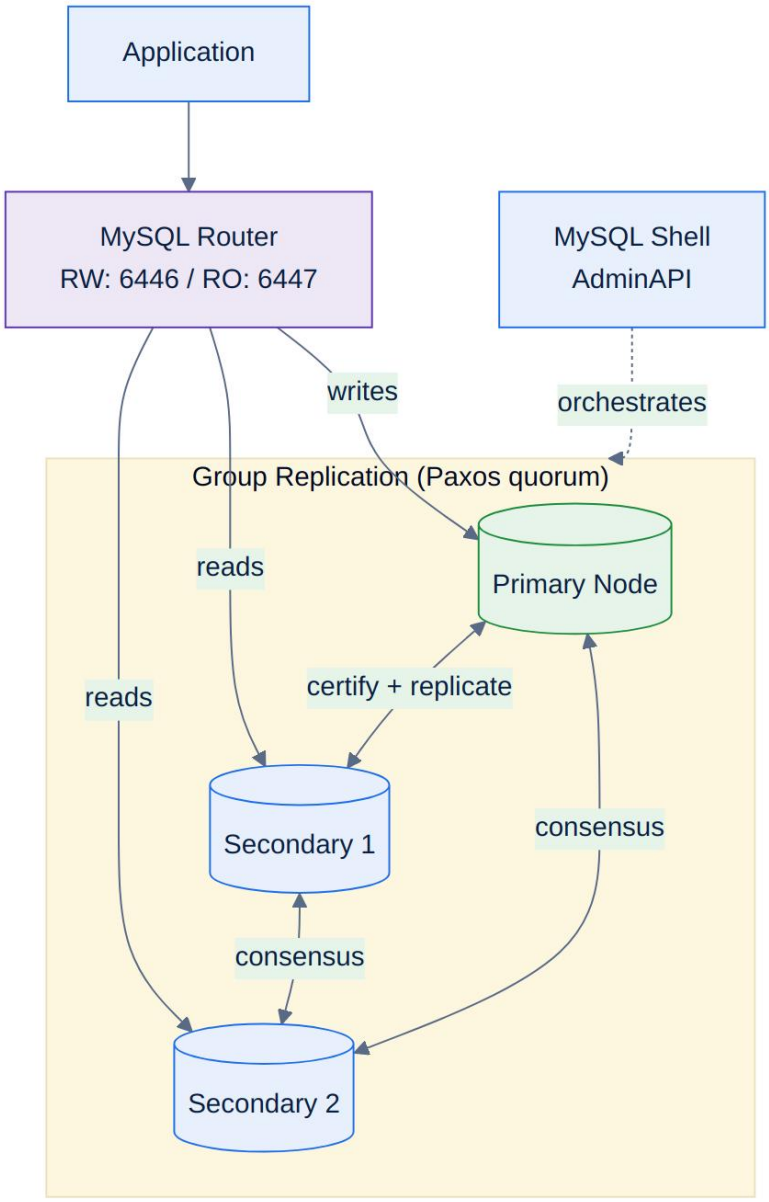


Figure 13.2 — InnoDB Cluster: Group Replication + Router + Shell

```
# Provision with MySQL Shell AdminAPI
mysqlsh
> dba.configureInstance('admin@node1:3306');   // repeat for node2, node3
> cl = dba.createCluster('prodCluster');
> cl.addInstance('admin@node2:3306');
> cl.addInstance('admin@node3:3306');
> cl.status();

# Bootstrap the Router against the cluster
mysqlrouter --bootstrap admin@node1:3306 --user=mysqlrouter
# Apps connect to Router: 6446 (R/W primary) and 6447 (R/O replicas)
```

13.3 HA Options Compared

Solution	Failover	Best for
Async replication + orchestrator/ProxySQL	Automated (external)	Read scale-out; flexible topologies.
Semi-sync replication	Manual/scripted	Bounded data loss without full consensus.
InnoDB Cluster (Group Replication)	Automatic	Standard self-healing HA on stock MySQL.
InnoDB ClusterSet	Automatic + regional	HA within a region plus DR across regions.
Galera / Percona XtraDB Cluster	Automatic (sync)	Multi-primary synchronous clusters.
NDB Cluster	Automatic	In-memory, shared-nothing, write-scale, 99.999%.

BEST PRACTICE

For most teams on stock MySQL 8.4, **InnoDB Cluster** (single-primary) plus **ClusterSet** for a DR region is the pragmatic default: automatic failover, transparent routing, and a supported management API.

14. Troubleshooting

Troubleshoot methodically: characterize the symptom, check the **error log** first, correlate with metrics and `SHOW ENGINE INNODB STATUS`, isolate the offending session/query, then remediate and confirm. Below are the highest-frequency production scenarios.

14.1 Scenario Playbook

Symptom	Likely cause	First moves
High CPU, slow app	Missing index / bad plan / scan	sys.statement_analysis; EXPLAIN; add index.
Threads_running spikes	Lock contention / thundering herd	SHOW ENGINE INNODB STATUS; find blocking trx.
Replica lag climbing	Single-threaded apply / big trx / slow disk	Enable MTS (replica_parallel_workers); check slow disk.
Too many connections	Pool leak / max_connections too low	Raise cap; fix app pool; kill idle sleepers.
Disk full on datadir	Binlogs / undo / temp growth	Purge binlogs; check long trx holding undo.
Crash + restart	OOM kill / hardware / bug	Error log; dmesg; verify InnoDB recovery completed.
Deadlocks	Inconsistent lock ordering	LATEST DETECTED DEADLOCK; standardize access order.

14.2 Find and Break a Blocking Chain

```
-- Who is blocking whom (8.0+, data_lock_waits)
SELECT r.trx_id AS waiting_trx, r.trx_mysql_thread_id AS waiting_thd,
       b.trx_id AS blocking_trx, b.trx_mysql_thread_id AS blocking_thd,
       r.trx_query AS waiting_query
FROM performance_schema.data_lock_waits w
JOIN information_schema.innodb_trx b ON b.trx_id = w.blocking_engine_transaction_id
JOIN information_schema.innodb_trx r ON r.trx_id =
w.requesting_engine_transaction_id;

-- Kill the blocker (confirm first!)
KILL 4821;   -- blocking_thd
```

14.3 Replication Health

```
SHOW REPLICA STATUS\G
-- Key fields: Replica_IO_Running / Replica_SQL_Running = Yes,
--             Seconds_Behind_Source, Last_Error, Retrieved/Executed_Gtid_Set

-- Skip a single poison transaction (GTID-based, use with care)
STOP REPLICA;
SET GTID_NEXT='<uuid>:<n>'; BEGIN; COMMIT; SET GTID_NEXT='AUTOMATIC';
START REPLICA;
```

WARNING
Skipping transactions can silently diverge a replica. Prefer to fix the root cause and, if in doubt, rebuild the replica from a fresh backup + GTID auto-positioning.

15. Common Errors

A reference of the errors DBAs hit most often, with root cause and resolution. Error numbers are the MySQL server codes shown to clients.

Error	Message (abridged)	Cause & fix
1045	Access denied for user	Wrong password/host/auth plugin. Verify account host mask; re-grant; check caching_sha2_password vs client support.
1040 / 1203	Too many connections	max_connections reached / user limit. Raise cap, fix connection-pool leak, kill idle sleepers.
2006	MySQL server has gone away	Idle > wait_timeout or packet > max_allowed_packet. Increase timeout/packet; add pool keepalive.
2013	Lost connection during query	Network drop, server crash, or timeout. Check error log + network; retry with backoff.
1213	Deadlock found; try restarting transaction	Circular lock waits. App should retry; standardize lock ordering; shorten transactions.
1205	Lock wait timeout exceeded	Blocked beyond innodb_lock_wait_timeout. Find blocker (14.2); shorten/commit sooner.
1062	Duplicate entry for key	Unique/PK collision. Use INSERT ... ON DUPLICATE KEY UPDATE or fix data.
1064	SQL syntax error	Malformed SQL / reserved word. Fix syntax; backtick-quote identifiers.
1114	The table is full	Temp/tablespace/disk limit. Free disk; raise tmp_table_size/limits; check autoextend.
1146	Table doesn't exist	Wrong schema, case sensitivity, or missing .ibd. Check lower_case_table_names; restore table.
1236	Could not find first log file (binlog)	Replica needs purged binlog. Rebuild replica from backup with GTID auto-positioning.
1032	Can't find record (replica)	Row missing on replica (drift/RBR). Reconcile data; rebuild replica if diverged.
28	No space left on device	Disk full. Purge binlogs/logs; extend volume; check long transactions holding undo.

BEST PRACTICE
When an error is cryptic, the **error log** plus `SHOW ENGINE INNODB STATUS\G` and `perror <n>` (e.g. `perror 28`) almost always reveal the real cause faster than guessing.

16. FAQs

Q: Is MySQL free?

The Community edition is free under GPL v2. Enterprise adds commercial features (backup, TDE keyring, audit, monitor) and support under a paid license.

Q: InnoDB or MyISAM?

InnoDB, effectively always. It provides ACID transactions, row-level locking, MVCC, crash recovery, and foreign keys. MyISAM is legacy and not crash-safe.

Q: What is the default isolation level?

REPEATABLE READ. InnoDB uses MVCC + gap locks to avoid most phantom rows even at this level. READ COMMITTED is common for high-concurrency OLTP.

Q: utf8 vs utf8mb4?

Use utf8mb4 (true 4-byte UTF-8, supports emoji and all code points). The legacy 'utf8' alias is only 3 bytes and should be avoided.

Q: Does MySQL support JSON?

Yes — a native JSON type with functions, JSON_TABLE, and functional/multi-valued indexes. For heavy document workloads, still evaluate PostgreSQL JSONB or a document store.

Q: How do I scale reads?

Add replicas behind a router/load balancer (ProxySQL, MySQL Router) and send read-only traffic to them, keeping writes on the primary.

Q: How do I scale writes?

Vertical scaling first; then application-level sharding, Vitess, or NDB Cluster. A single InnoDB primary is the write bottleneck.

Q: When should I shard?

Only after exhausting indexing, caching, read replicas, and vertical scaling — typically past single-node write or storage limits (often multi-TB / very high write QPS).

Q: MySQL vs MariaDB — can I switch freely?

They share heritage but have diverged (auth, JSON, optimizer, Galera). Migrations are usually feasible but require testing; they are not drop-in identical at 8.x/11.x.

Q: How big can an InnoDB table get?

Effectively very large (tens of TB), bounded by tablespace and OS limits. Practical limits come from operational concerns (backup/restore time, index maintenance), not the format.

17. Interview Questions

Grouped by theme with concise, senior-level answers — useful for DBA, cloud, and data-engineering interviews.

17.1 Fundamentals

Question	Answer
Explain MySQL's architecture layers.	Connection layer (auth/threads), SQL layer (parser, optimizer, executor), pluggable storage-engine layer (InnoDB), and physical storage (tablespaces, redo/undo, binlog).
What makes InnoDB ACID?	Redo log (write-ahead logging) for durability + crash recovery, undo logs for atomicity/rollback and MVCC, row locking + isolation levels for consistency & isolation.
Clustered vs secondary index?	InnoDB stores the table in a clustered index on the PK; secondary indexes store the PK as the row pointer, so a secondary lookup may need a second (bookmark) lookup.
What is MVCC?	Multi-Version Concurrency Control: readers see a consistent snapshot via undo-log versions instead of blocking writers, enabling non-locking consistent reads.

17.2 Performance

Question	Answer
How do you find a slow query?	Slow query log + pt-query-digest, or sys.statement_analysis / statements_with_full_table_scans, then EXPLAIN ANALYZE the offender.
Left-prefix rule?	A composite index (a,b,c) can serve predicates on a, a+b, a+b+c — but not b or c alone; column order matters.
Covering index?	An index containing every column a query needs, so it's answered from the index without touching the table (Using index in EXPLAIN).
Buffer pool sizing?	~60-75% of RAM on a dedicated host; verify with a >99% buffer-pool hit ratio.

17.3 Replication, HA & Backup

Question	Answer
GTID vs file/position replication?	GTID assigns each transaction a global ID, enabling auto-positioning and reliable failover/re-pointing; file+position is manual and error-prone.
Async vs semi-sync vs Group Replication?	Async = no wait (risk of loss); semi-sync = wait for >=1 replica ack (bounded loss); Group Replication = Paxos consensus (synchronous certification).
Difference between HA and DR?	HA survives node failure within a site (fast, automatic); DR survives site/region loss and is defined by RPO/RTO with cross-region copies.

Question	Answer
How does PITR work?	Restore the last full backup, then replay binary logs with mysqlbinlog up to a chosen timestamp/position just before the incident.
Backups vs replication?	Replication propagates logical corruption/DROPs instantly; only backups + PITR recover from bad data. They are complementary, not interchangeable.

18. Real-world Use Cases

Domain	Why MySQL fits	Typical pattern
Web & SaaS OLTP	Fast reads/writes, mature ecosystem, cloud-native	Primary + read replicas behind Router/ProxySQL; utf8mb4; InnoDB.
E-commerce	Transactions for orders/inventory, FKs, ACID	InnoDB Cluster for HA; read replicas for catalog; PITR for order safety.
Multi-tenant SaaS	Schema-per-tenant or shared-schema flexibility	Row-level tenant_id + composite indexes; per-tenant backup/export.
Financial services	ACID + auditability + encryption	TDE at rest, TLS, audit log, strict RBAC; DR ClusterSet; SOX/PCI evidence.
Content management	Simple relational model, huge read scale	Heavy caching + read replicas; JSON columns for flexible metadata.
Gaming / leaderboards	High write throughput, low latency	Sharded InnoDB or MEMORY/Redis tier for hot counters; async replicas.
Analytics offload	Keep OLTP clean; analyze elsewhere	CDC/binlog to warehouse (HeatWave, StarRocks, Snowflake); replica for reporting.

NOTE

The recurring production pattern is **one InnoDB primary for writes, several replicas for reads, a router for failover, encrypted tested backups for recovery, and a DR region** — adjust the ratios to the workload.

19. Best Practices

19.1 Schema & Query

Design

- Every table has a compact, monotonic primary key (avoid random UUID PKs as clustered keys).
- Choose the narrowest correct data types; use utf8mb4 with the 0900 collation.
- Index for your real query patterns; prefer covering & composite indexes; drop unused ones.
- Keep transactions short; never leave one open across user think-time.

19.2 Configuration & Operations

Run it well

- Size the buffer pool to the working set; disable THP; use O_DIRECT and NVMe.
- Keep structural config in version-controlled my.cnf; use SET PERSIST for dynamic tuning.
- Enable GTID + row-based binlog for safe replication and PITR.
- Standardize on 8.4 LTS; apply quarterly Critical Patch Updates.

19.3 Security, Backup & HA

Protect & recover

- Least-privilege roles, per-host accounts, TLS everywhere, encryption at rest, audit logging.
- Automate encrypted offsite backups and, above all, test restores regularly.
- Deploy HA (InnoDB Cluster) and DR (ClusterSet) sized to documented RTO/RPO; rehearse failover.
- Monitor golden signals with alerting; review slow queries and capacity trends weekly.

20. References

20.1 Official Documentation

- MySQL 8.4 Reference Manual — dev.mysql.com/doc/refman/8.4/en/
- MySQL 9.x Reference Manual (Innovation) — dev.mysql.com/doc/refman/9.7/en/
- InnoDB Storage Engine & Architecture — MySQL Reference Manual, Ch. 17
- Group Replication & InnoDB Cluster / ClusterSet — MySQL Reference Manual + MySQL Shell AdminAPI docs
- MySQL Release Model (Innovation & LTS) — dev.mysql.com/doc/refman/8.4/en/mysql-releases.html
- Lifecycle & EOL dates — endoflife.date/mysql

20.2 Tools

- Percona XtraBackup & Percona Toolkit (pt-query-digest) — docs.percona.com
- Percona Monitoring & Management (PMM) — percona.com/software/pmm
- Prometheus mysqld_exporter + Grafana — github.com/prometheus/mysqld_exporter
- ProxySQL — proxysql.com

20.3 Recommended Reading

- High Performance MySQL (O'Reilly) — Schwartz, Zaitsev, et al.
- MySQL 8 Administrator's Guide — Packt
- Oracle MySQL Blog & Percona Blog — release notes, tuning deep-dives

— End of Knowledge Base —